

RDSI

Representation, Domain Modelling and Software Implementation

Diophantine equations in which only the unit q represents a cycle

Bilal El Issaoui, Amsterdam, 2026

1. The core in three sentences

In a Diophantine equation $N = c_1 \cdot A + c_2 \cdot B$ we look for integer solutions. If q represents a cycle, the question is not "what is the remainder?" but "how many complete cycles have we had, and where in the cycle are we now?"

By using $25 \equiv 1 \pmod{12}$ you force the equation to see the cycle as an endless succession from 1 to 12, not from 0 to 11. This changes the meaning of the coefficients (c_1, c_2) in a way that fits natural cycles exactly: the clock with hours 1 to 12, the months 1 to 12, the days of the month.

2. Why this research is needed

Almost every cycle in software today is modelled 0-based: positions 0 up to and including $q - 1$. This seems neutral, but it causes a structural problem. The value that marks the end of the cycle is at the same time the value of the beginning. On the clock, 24:00 is both the end of the day and the beginning of the night. Software developers know the consequences as off-by-one errors: do we count to 23 or to 24? Is the boundary included or not?

This note shows that the choice between $p \equiv 0$ and $p \equiv 1 \pmod{q}$ is not an arithmetical choice but a representational choice. The arithmetic stays the same. The domain model does change, and that difference is measurable in code, in tests and in the number of special cases in the code (§5.7).

3. What this note proves

- The (25,12) system is a full-fledged Diophantine system with its own foundation relation, its own boundary values and its own generation method (the ladder).
- The clock [3600, 60, 1] is likewise Diophantine, but built on an opposite mechanism. Both are correct. They serve different purposes.
- The transition $13 \equiv 1 \pmod{12}$, which everyone knows from the clock, is not a loose rule of arithmetic but the formal heart of every cycle with $p \equiv 1 \pmod{q}$.

4. Structure

Part	Content
§2	The Diophantine basis: the equation, the foundation and the ladder
§3	The cycle: why 1 to 12 is structurally different from 0 to 11
§4	Two architectures: (25,12) alongside the clock [3600, 60, 1]
§5	From theory to code: examples, pseudocode and tests
§6	The defence and the summary

§2 The Diophantine basis

$$N = c_1 \cdot A + c_2 \cdot B, \text{ with } A \text{ and } B \text{ integer and non-negative}$$

2.1 The foundation relation

The system rests on a single relation: $c_1 \equiv 1 \pmod{c_2}$. For the (25,12) system that is $25 \equiv 1 \pmod{12}$. Because $12 \cdot B \equiv 0 \pmod{12}$, B drops out of the equation modulo 12 entirely:

$$N = 25 \cdot A + 12 \cdot B \Rightarrow N \equiv A \pmod{12} \Rightarrow A \sim 0 \sim N \bmod 12$$

The smallest coefficient A_0 is therefore determined directly in one single step. That is the engine of the whole system. Compare the clock: there nothing disappears in one step; each layer has to be calculated separately.

2.2 Worked example: $N = 500$

Step 1: $A_0 = 500 \bmod 12 = 8$. Step 2: the ladder of all solutions.

k	$A_k = A_0 + 12k$	$B_k = (500 - 25A_k) / 12$	Check: $25A_k + 12B_k$
0	8	25	$200 + 300 = 500$
1	20	0	$500 + 0 = 500$
2	32	-25	invalid, B is negative

$N = 500$ therefore has exactly two representations: $(A,B) = (8,25)$ and $(20,0)$.

2.3 The boundary values of the system

Concept	Value	Meaning
Frobenius number	263	the largest number that cannot be written as $25A + 12B$
Foundation point	$264 = 24 \cdot 11$	from here on, for every number: $A_0 = N \bmod 12$
Ladder step	+12 in A, -25 in B	from one solution all others follow
Number of solutions	$R(N)$	counts how many pairs (A,B) belong to N; can be zero

2.4 What this means

Because the coefficients are unbounded, every number N gets a whole spectrum of representations. The system therefore answers not with a single number but with a structured family: the ladder. This is the opposite of the clock, which removes all freedom precisely in order to guarantee one unique answer. Both mechanisms are Diophantine. They deliberately produce opposite outcomes.

§3 The cycle: 1 to 12 instead of 0 to 11

3.1 The contradiction of the 0-based model

In the usual model the positions run from 0 up to and including 11. The next position after 11 is 0 again. As a result the number 12, the end of the cycle, is at the same time position 0, the beginning of the cycle. End and beginning coincide in a single value. For a machine that is acceptable. For a domain model it is a source of confusion, because in the real world end and beginning are different moments. Midnight is not the same moment as the end of the day.

3.2 The same number, two models

Number T	Model 0 to 11	Model 1 to 12
12	position 0, called "beginning"	position 12, the end of the cycle
13	position 1	position 1, after one transition
24	position 0: end and beginning at once	position 12: exclusively end
25	position 1	position 1, after two complete cycles

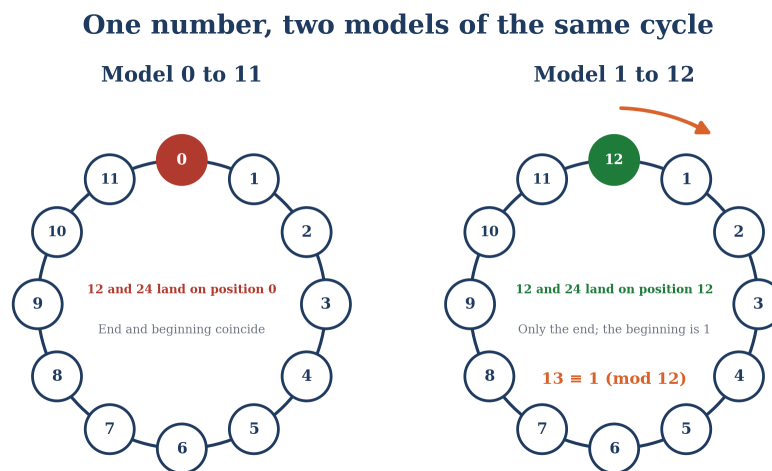


Figure 1. The same cycle in two models. In model 0 to 11 the top position 0 is end and beginning at once; in model 1 to 12 position 12 is only the end, and 13 returns to 1.

3.3 The two questions of every cycle

For $T = 25$ in the $(25, 12)$ system, a single calculation answers both questions:

- **Where in the cycle are we?** $A_0 = 25 \bmod 12 = 1$. Position 1 of the cycle 1 to 12.
- **How many complete cycles have elapsed?** $25 = 2 \cdot 12 + 1$. Two complete cycles have passed.

Position 0 never occurs. There is no hour 0: that is hour 12. And hour 13 is hour 1. The position follows from $((T - 1) \bmod 12) + 1$. $A_0 = T \bmod 12$ is the smallest coefficient in the ladder and can be 0 (for $T = 12$ or 24), while the position is then 12.

In the model 0 to 11 you get the same remainder 1, but the number of complete cycles remains a by-product of the division. In the model 1 to 12 the transition itself is the starting point: $p \equiv 1 \pmod{q}$ formally determines at which moment the cycle rolls over. The transition is no longer a consequence, but the heart of the model.

3.4 The transition everyone knows

On the 12-hour clock the hand moves on from 12 to 1. In numbers: $13 \equiv 1 \pmod{12}$. Everyone uses this transition daily. This note makes that transition explicit as a mathematical principle. Every p with $p \equiv 1 \pmod{q}$ generates such a cycle structure of its own.

Pair (p, q)	Relation	Where you know it from
(13, 12)	$13 \equiv 1 \pmod{12}$	the 12-hour clock, hours 1 to 12
(10, 9)	$10 \equiv 1 \pmod{9}$	the digital root via the digit sum
(25, 12)	$25 \equiv 1 \pmod{12}$	this work: foundation of the (25,12) system

Conclusion: the choice $p \equiv 1 \pmod{q}$ is a representational choice. The outcome of the calculation does not change. The model does change: end and beginning are separated, and the transition from one cycle to the next becomes formal.

§4 Two architectures, two purposes

The (25,12) system and the clock [3600, 60, 1] are both linear Diophantine representation systems. They cannot be compared in terms of purpose: the one aims at an ordered family of representations, the other at exactly one representation per moment. The clock is therefore not a failed Diophantine system. The clock is a correct system with a different purpose.

Property	(25,12) system	Clock [3600, 60, 1]
Origin	designed, the coefficients are chosen	known since antiquity, Babylonian sexagesimal
Purpose	counting and generating representations	one unique time indication per moment
Structural condition	$c_1 \equiv 1 \pmod{c_2}$	each base is a multiple of the next
Limits on coefficients	none	yes: seconds and minutes below 60, hours below 24
Number of representations	variable, zero or more (the ladder)	exactly one per instant T
Role of mod	global: $A_0 = N \bmod 12$ determines everything in one step	local: each layer has its own modulus 60, 60, 24
Frobenius number	263, the largest non-representable number	none, every instant is representable
Transitions	via the ladder: A grows by 12, B falls by 25	via carry and overflow within the chain

4.1 Why $3600 \equiv 0 \pmod{60}$ is precisely correct

The foundation relation $c_1 \equiv 1 \pmod{c_2}$ does not hold for the clock, and that is deliberate. The divisibility 3600, 60, 1 guarantees that the positions never overlap. Thereby every decomposition is unique. The foundation relation does the opposite: it makes multiple decompositions possible and structures them into a ladder. Two mechanisms, two outcomes, each suited to its own purpose.

4.2 The trap of the 12-hour analogy

The 12-hour dial, where 13:00 equals 1:00, shares the modularity idea with the (25,12) system. That is the metaphor. The structural clock [3600, 60, 1] does not share that idea: it achieves uniqueness through the divisibility chain. Whoever confuses the metaphor with the structure misses precisely the point this note makes.

4.3 Large numbers become readable

Large number	Readable form	Layers
45,296,789 milliseconds	12:34:56.789	days, hours, minutes, seconds, milliseconds
3,661 seconds	1:01:01	hours, minutes, seconds
1,234,567 bytes	1.23 megabytes	bytes, kilobytes, megabytes
20260917	17 September 2026	year, month, day

Positional systems make large numbers readable by dividing them into layers. The (25,12) system makes the reverse movement: from one number it generates an ordered family of representations. Reading and generating are two different purposes, and the two architectures fit them precisely.

§5 From theory to code

5.1 Why this is measurable in practice

1. **Readability.** One line per layer turns 45,296,789 into the time 12:34:56.789.
2. **Edge cases disappear.** "Three hours from 11 o'clock" is a single mod calculation as soon as the representation is right.
3. **Testable invariants.** The foundation $A_0 = N \bmod 12$ is a hard property that can be recorded directly as a test in the build pipeline.

5.2 Pseudocode: the ladder generator

```
function decompose(N):
  A0 ← N mod 12                // foundation, one global step
  k ← 0
  repeat:
    A ← A0 + 12 * k
    B ← (N - 25 * A) / 12
    if B < 0: stop
    add (A, B) to the ladder
    k ← k + 1
  return the ladder            // length is R(N)
```

5.3 Pseudocode: the clock as a cascade

```
S ← T mod 60
M ← (T div 60) mod 60
H ← (T div 3600) mod 24
D ← T div 86400          // unbounded
Ms ← T mod 1000
```

Note the difference: the ladder generator solves the problem in one global step and then generates everything. The clock solves the problem layer by layer and gives exactly one answer.

5.4 Worked example: $T = 45,296,789$ milliseconds

Layer	Calculation	Value	Limit
Days	$T \text{ div } 86,400,000$	0	unbounded
Hours	$(T \text{ div } 3,600,000) \bmod 24$	12	0 to 23
Minutes	$((T \bmod 3,600,000) \text{ div } 60,000)$	34	0 to 59
Seconds	$((T \bmod 60,000) \text{ div } 1000)$	56	0 to 59
Milliseconds	$T \bmod 1000$	789	0 to 999

Check: $3,600,000 \cdot 12 + 60,000 \cdot 34 + 1000 \cdot 56 + 789 = 45,296,789$. The sum holds.

5.5 Four tests you can record directly

Property	Test in words
Foundation	for every number N from 264 onwards, the smallest A equals $N \bmod 12$
Ladder holds	every pair on the ladder satisfies $25A + 12B = N$ and B is non-negative
Uniqueness of the clock	decomposing after composing always returns the original triple
Transition	after position 12 comes position 1 with one extra cycle, never a position 0 as beginning: hour 0 does not exist (that is 12) and hour 13 is 1

5.6 Test results

The four tests were implemented in a Python reference implementation of the ladder generator and the clock cascade (rdsi_tests.py) and executed. All four passed.

Test	What was checked	Result
Foundation	every N from 264 to 200,000; 263 is the largest non-representable number	passed
Ladder holds	every N from 0 to 5,000, compared with an independent brute-force count; N = 500 gives (8,25) and (20,0)	passed
Uniqueness of the clock	a sweep over two days and 200,000 random times; 45,296,789 ms gives 0 d 12:34:56.789	passed
Transition	T = 1 to 100,000: position always 1 to 12, never 0; 12 gives 12, 13 gives 1, 24 gives 12, 25 gives 1; after position 12 comes position 1 with one extra cycle	passed

5.7 Comparison: 0-based versus 1-based

An independent oracle simulates the cycle step by step, without any formula. Three operations are compared against it: the label at step T, the label after n steps, and the number of end-to-beginning transitions. Three implementations were run for $q = 7, 9, 12, 24$ and 60 (rdsi_compare.py): the usual 0-based model with plain mod arithmetic, the 0-based model with the special cases needed to make it correct, and the 1-based model with $p \equiv 1 \pmod{q}$.

Implementation ($q = 12$)	Label wrong	Add wrong	Transitions wrong	Branches
0-based, plain mod	8.3%	8.3%	14.9%	0
0-based, corrected	0%	0%	0%	6
1-based, $p \equiv 1 \pmod{q}$	0%	0%	0%	0

Plain 0-based arithmetic gives a wrong label at every boundary value (12, 24, 36, ...), a share of $1/q$ of all inputs: from 14.3% for $q = 7$ down to 1.7% for $q = 60$. Making the 0-based model correct costs six conditional branches. The 1-based model is correct on every tested input with none. Both correct models agree with the oracle exactly, for all five values of q . A branch is an if statement, a ternary or an and/or condition.

Result: the 1-based representation removes all six special cases of the 0-based model and stays correct on every input.

§6 The defence

6.1 The possible logical criticism

The possible logical criticism might read:

" $13 \equiv 1 \pmod{12}$ is just standard arithmetic. The clock simply goes back to 1, doesn't it? There is nothing new in that."

6.2 The answer

"That is entirely correct. Mathematically that is the standard. My work is not about the arithmetical rule, but about the representation of cyclic domains. The standard notation with $24 \equiv 0$ is based on counting from 0. As a result, 24:00 is at the same time the end and the beginning, and in software this leads to well-known off-by-one errors.

By using $25 \equiv 1 \pmod{12}$ I formalise a cycle from 1 to 12. The focus shifts from the remainder 0 to the transition $p \equiv 1 \pmod{q}$. The transition is thereby no longer a by-product of the calculation, but the formal heart of the cycle. That is the innovation: a mathematical reformulation of the model, not of the arithmetical outcome."

6.3 Why this position is strong

- The arithmetic remains unchanged. The claim concerns the model, not the outcome. There is nothing to argue against there.
- The foundation relation $c_1 \equiv 1 \pmod{c_2}$ appears in the literature as a description. This work uses it as a design rule. That reversal is the contribution.
- The ladder $A_k = A_0 + 12k$ generates all solutions. Existing methods only count.
- Every pair (p, q) with $p \equiv 1 \pmod{q}$ works this way. The digital root at $(10, 9)$ is the well-known example.
- The comparison with the clock $[3600, 60, 1]$ shows precisely why $3600 \equiv 0 \pmod{60}$ is no defect there: two mechanisms, two purposes.
- The difference is measured, not asserted: the 0-based model needs six special-case branches to be correct, the 1-based model none (§5.7).

6.4 Summary in three points

1. **Representation.** Only the unit q represents the cycle. With $p \equiv 1 \pmod{q}$ the transition becomes the formal heart instead of a by-product.
2. **Domain modelling.** The $(25, 12)$ system and the clock are both Diophantine and structurally different: multiple representation with unbounded coefficients versus unique representation with bounded coefficients.
3. **Software implementation.** The theory translates directly into code: one global step for A_0 , a ladder generator for all solutions, and four hard tests as invariants.

6.5 Relation to earlier work

Work	Contribution
The clock [3600, 60, 1], another Diophantine structure (2026)	distinction between foundation relation and positional divisibility
Time decomposition of the clock [3600, 60, 1] (2026)	didactic route and decomposition down to the millisecond
The formula with $19 \equiv 1 \pmod{9}$ (2026)	convergence of digital reduction to one
This note (RDSI)	the representation choice $p \equiv 1 \pmod{q}$ as a domain and software principle

The clock has shown for thousands of years how cycles work. This note makes explicit which mathematical choice lies within it, and shows how to record that choice deliberately, testably and without doubt in software.